

Dynamic Hedge Ratio Pipeline Orchestrator

dynamic_beta.py — Mathematical Notes & Documentation

OLS Initialization, Kalman State Tracking, RTS Smoothing, and Z-Score Signaling

Project Dark-Wing | July 26, 2026

Abstract. Practical statistical arbitrage demands a cohesive execution pipeline. Rather than operating isolated models, `dynamic_beta.py` acts as the primary orchestrator that chains several modules: static Ordinary Least Squares (OLS) initialization, Ornstein-Uhlenbeck (OU) mean-reversion analysis, Kalman Filter state tracking, RTS smoother optimization, and rolling Z-score generation. Additionally, it implements a rolling CUSUM-style variance regime-detection test to alert downstream execution modules of structural breaks in the hedge ratio. This document reviews the mathematical details of the parameter heuristics, pipeline flow, and structural break checking.

Pipeline Architecture & Components

In high-frequency pairs trading, estimating the dynamic hedge ratio is prone to tracking lag and noise sensitivity. The orchestrator mitigates this by layering estimation filters in sequence:

OLS $\rightarrow$ OU Half-Life $\rightarrow$ Q/R Heuristic $\rightarrow$ Kalman Filter $\rightarrow$ RTS Smoother $\rightarrow$ Z-Score

Each component plays a specific role:

1. **SpreadBuilder (OLS):** Runs a static baseline regression to obtain initial beta (β_0) and standard residual variance.
2. **HalfLife (OU):** Analyzes the OLS residuals to extract the half-life of mean reversion (τ). This window size is crucial for setting downstream rolling periods.
3. **Heuristic Engine:** Dynamically sets the process noise Q and measurement noise R to prevent covariance collapse.
4. **KalmanFilterPairs (Causal):** Evaluates dynamic filtering hedge ratios $\hat{\beta}_{t|t}$ for live trading signals.
5. **KalmanSmoother (Non-Causal):** Performs RTS backward smoothing to produce $\hat{\beta}_{t|T}$ for historical backtest calibration.
6. **ZScore:** Generates normalised entry/exit signals on the dynamic spread.

The Q/R Parameter Heuristic

The performance of the Kalman Filter hinges on the ratio of the process noise covariance Q to the measurement noise covariance R .

- Q too large relative to R : The filter overreacts to price shocks, causing the hedge ratio $\hat{\beta}_{t|t}$ to fluctuate erratically, increasing transaction costs.
- R too large relative to Q : The filter ignores the time-varying nature of the relationship, converging to a static OLS model.

To provide robust initial conditions before MLE optimization, we derive a noise heuristic using the Ornstein-Uhlenbeck (OU) process properties of the spread:

$$dS_t = -\kappa(S_t - \theta)dt + \sigma dW_t$$

where κ is the speed of mean reversion, $\tau_{\text{half-life}} = \ln(2)/\kappa$, and σ is the residual volatility.

Q/R Noise Heuristics

Under the assumption that the spread follows an OU process, the process variance Q of the hedge ratio scales inversely with the half-life:

$$Q \approx \frac{\sigma_{\text{resid}}^2}{\tau_{\text{half-life}}} \quad (1)$$

where σ_{resid}^2 is the variance of the residuals of the OU process. The measurement noise covariance R is initialized to the total variance of the static OLS spread:

$$R \approx \text{Var}(S_t^{\text{OLS}}) \quad (2)$$

The Complete Fitting Pipeline

The orchestrator executing `DynamicBeta.fit()` steps through the following sequence:

Phase 1: Baseline Static Regression

First, we run static OLS:

$$Y_t = \alpha_0 + \beta_0 X_t + e_t$$

This provides the prior hedge ratio β_0 . The residuals $S_t^{\text{OLS}} = Y_t - \beta_0 X_t$ form the initial spread.

Phase 2: Half-Life Estimation

We discretize the spread as an AR(1) process:

$$S_t^{\text{OLS}} - S_{t-1}^{\text{OLS}} = a + bS_{t-1}^{\text{OLS}} + \eta_t$$

We estimate b via OLS, which yields the speed of mean reversion:

$$\kappa = -\ln(1 + b)$$

and the half-life of mean reversion:

$$\tau_{\text{half-life}} = \frac{\ln(2)}{\kappa}$$

Phase 3: Kalman State Update

We initialize the Kalman Filter with $\hat{\beta}_{0|0} = \beta_0$ and $P_{0|0} = 1000.0$. We set Q and R from the heuristics in Eq. (1) and Eq. (2). We run the filter over the log prices, yielding the filtered states:

$$\hat{\beta}_{t|t}, \quad P_{t|t}, \quad \text{and} \quad S_t = Y_t - \hat{\beta}_{t|t}X_t$$

Phase 4: RTS Smoothing

For backtest calibration, we run the RTS backward smoother to obtain:

$$\hat{\beta}_{t|T}, \quad P_{t|T}, \quad \text{and} \quad S_t^{\text{smooth}} = Y_t - \hat{\beta}_{t|T}X_t$$

Phase 5: Signal Normalization

The dynamic spread S_t is normalized using a rolling Z-score with window size $W = 2\tau_{\text{half-life}}$:

$$Z_t = \frac{S_t - \mu_{\text{roll}}(S_t, W)}{\sigma_{\text{roll}}(S_t, W)}$$

The strategy generates buy/sell signals on Z_t .

Regime Detection Algorithm

In statistical arbitrage, a critical risk is cointegration breakdown (the assets diverge permanently due to structural reasons). This manifests as a level shift in β_t . We detect this via a rolling CUSUM-style test on the hedge ratio.

Structural Break Detection

Let μ_{global} be the mean of β_t over the entire historical window. For each timestep t , we compute the rolling average $\mu_{\text{roll}}(t, W)$ and rolling standard deviation $\sigma_{\text{roll}}(t, W)$ over a lookback window $W = 126$ (6 months). A regime break is flagged if the deviation exceeds a statistical threshold:

$$|\mu_{\text{roll}}(t, W) - \mu_{\text{global}}| > K \cdot \sigma_{\text{roll}}(t, W) \quad (3)$$

where K is the sensitivity parameter (default $K = 2.0$).

When a regime break is flagged, it signals that the historical cointegration relation has broken down, and trade execution should reduce position sizes or halt entries.

Worked Numerical Example

Pipeline Initialization Example

Let Y and X be log-prices with $N = 252$ days.

1. **OLS Result:** $\beta_0 = 1.25$, $\text{Var}(S^{\text{OLS}}) = 0.04$.
2. **OU Fit:** Speed of mean reversion $\kappa = 0.055$ per day.
3. **Half-life:** $\tau_{\text{half-life}} = \ln(2)/0.055 = 12.6 \approx 13$ days.
4. **OU Residual Variance:** $\sigma_{\text{resid}}^2 = 0.0013$.

Using these estimates, the orchestrator sets:

$$R = 0.04$$

$$Q = \frac{0.0013}{13} = 1.0 \times 10^{-4}$$

The Kalman Filter is initialized with:

$$\hat{\beta}_{0|0} = 1.25, \quad P_{0|0} = 1000.0$$

The rolling Z-score window is set to:

$$W = 2 \times 13 = 26 \text{ days}$$

This prevents arbitrary window selection and links signal generation directly to the underlying mean-reversion speed.

Class Structure & Flowchart

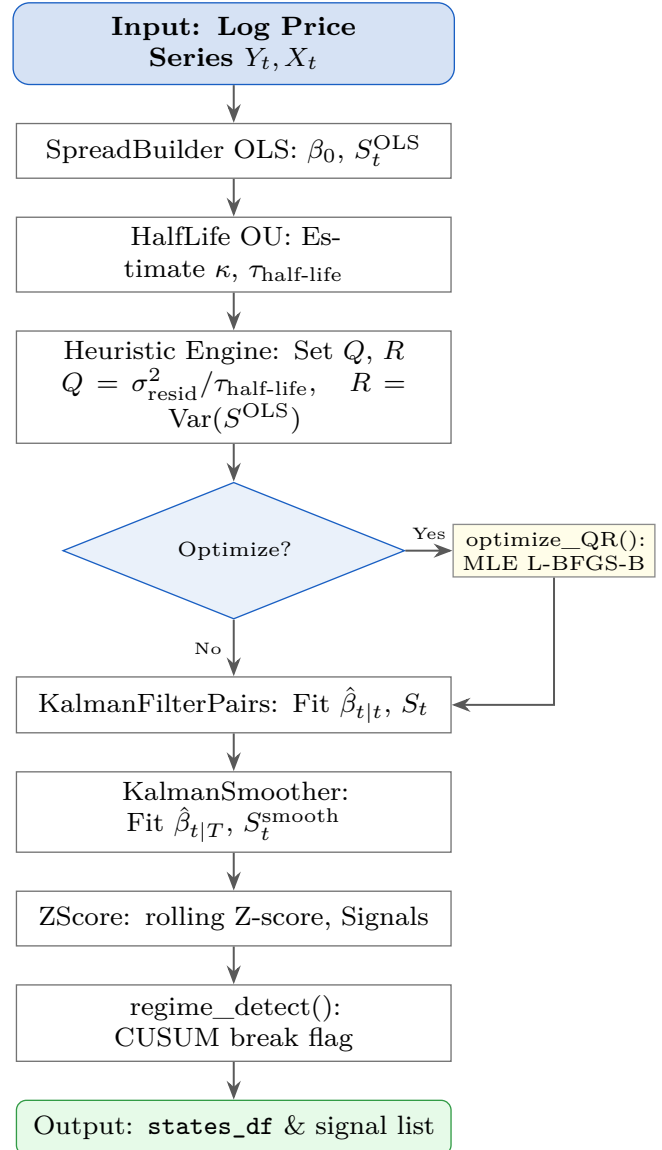


Figure 1: Process flow of the DynamicBeta pipeline.

Python Implementation Snippets

dynamic_beta.py Main Architecture

```

class DynamicBeta:
    def __init__(self, y, x, Q=None, R=None, P_init=1000.0,
                  beta_init=None, optimize=False,
                  half_life_window=252, entry_z=2.0,
                  exit_z=0.5, stop_z=3.5, run_smoother=True):
        self._y = y; self._x = x
        self.P_init = P_init
        self.optimize = optimize
        self.entry_z = entry_z; self.exit_z = exit_z;
        self.stop_z = stop_z
        self.run_smoother_flag = run_smoother

    # Step 1: OLS baseline
    self._sb = SpreadBuilder(y=self._y, x=self._x,
                             mode="ols").build()
    ols_beta = self._sb.get_beta()
    self._ols_spread = self._sb.spread_series()

    # Step 2: Half-Life
    _hl = HalfLife(self._ols_spread).estimate()
    self._tau = _hl.half_life_days

    # Step 3: Heuristics
    _ou_resid_var = np.var(_hl.get_ou_residuals())
    self._Q = Q if Q is not None else max(1e-8,
        _ou_resid_var/self._tau)
    self._R = R if R is not None else np.var(self._ols_spread.values)
    self._beta_init = beta_init if beta_init is not None else ols_beta

    def fit(self):
        self._kf = KalmanFilterPairs(Q=self._Q, R=self._R,
                                     beta_init=self._beta_init,
                                     P_init=self.P_init)

        if self.optimize:
            self._kf.optimize_QR(self._y, self._x)
        self._states_df = self._kf.filter(self._y, self._x)

        if self.run_smoother_flag:
            self._ks = KalmanSmoother(self._kf)
            self._ks.smooth()
            self._states_df["smoothed_beta"] = self._ks._smoothed_beta
            self._states_df["smoothed_spread"] = self._ks.smoothed_spread().values

        kalman_spread = self._kf.spread_series()
        zs_window = max(10, int(self._tau * 2))
        self._zscore = ZScore(kalman_spread, window=zs_window, mode="rolling",
                               entry_z=self.entry_z,
                               exit_z=self.exit_z,
                               stop_z=self.stop_z)

        self._zscore.compute()
        self._zscore.generate_signals()

        self._states_df["zscore"] = self._zscore.zscore_series().values
        self._states_df["signal"] = self._zscore.signal_series().values
        self._fitted = True
        return self._states_df

```

The EM Algorithm for Variance Tuning

While direct Maximization of Log-Likelihood (MLE) using Scipy's L-BFGS-B is robust, the Expectation-Maximization (EM) algorithm provides an elegant, derivative-free iterative framework for estimating the noise covariances Q and R .

The EM algorithm treats the hidden hedge ratios $\beta_{1:T}$ as latent variables and alternates between:

- **E-step:** Run the Kalman Filter and RTS Smoother using current parameter guesses $Q^{(j)}, R^{(j)}$ to compute the smoothed state expectations $\mathbb{E}[\beta_t | \mathcal{F}_T] =$

$\hat{\beta}_{t|T}$ and covariances $P_{t|T}$.

- **M-step:** Maximize the expected log-likelihood of the complete data relative to Q and R .

EM Update Equations

The updated process variance $Q^{(j+1)}$ and measurement variance $R^{(j+1)}$ are computed from the smoothed states as:

$$Q^{(j+1)} = \frac{1}{T-1} \sum_{t=2}^T [(\hat{\beta}_{t|T} - \hat{\beta}_{t-1|T})^2 + P_{t|T} + P_{t-1|T} - 2P_{t,t-1|T}] \quad (4)$$

$$R^{(j+1)} = \frac{1}{T} \sum_{t=1}^T [(Y_t - \hat{\beta}_{t|T} X_t)^2 + X_t^2 P_{t|T}] \quad (5)$$

where the lag-1 covariance is obtained via:

$$P_{t,t-1|T} = P_{t|T} \Xi_{t-1}$$

The EM algorithm converges monotonically to a local maximum of the likelihood function. It is particularly stable when the initial parameter guesses derived from the OU heuristics in Eq. (1) and Eq. (2) are used as starting values.

Signal Threshold Optimization

In the ZScore normalization phase, setting the entry threshold Z_{entry} (e.g., 2.0) and exit threshold Z_{exit} (e.g., 0.5) determines the trade frequency and win rate.

Under the assumption of a stationary de-meant Gaussian spread $S_t \sim \mathcal{N}(0, \sigma^2)$, the probability that the spread deviates beyond Z_{entry} standard deviations is:

$$P(|Z| > Z_{\text{entry}}) = 2(1 - \Phi(Z_{\text{entry}}))$$

For $Z_{\text{entry}} = 2.0$, this probability is approximately 4.56%. The expected time between entry signals is proportional to the half-life:

Expected Trade Frequency

$$\mathbb{E}[T_{\text{wait}}] \propto \frac{\tau_{\text{half-life}}}{P(|Z| > Z_{\text{entry}})} \quad (6)$$

If the assets' half-life $\tau_{\text{half-life}}$ is short, the spread reverts rapidly, allowing for frequent round-trip trades. If $\tau_{\text{half-life}}$ is long, the trade frequency drops, and capital is locked up for longer durations, requiring wider entry thresholds to ensure adequate margin of safety.

Interview Preparation

Key Pipeline Orchestration Questions

1. **Why do we use OLS regression to initialize the Kalman Filter?**

The Kalman Filter requires a prior state mean $\beta_{0|0}$. Using OLS provides a statistically optimal prior baseline, accelerating the filter convergence.

2. **How does the OU half-life help size the Z-score lookback window?**

The half-life represents the average duration of a deviation. Sizing the window to 2τ ensures the rolling mean and standard deviation capture both the deviation and reversion phases.

3. **What is the financial danger of ignoring a regime break?**

If cointegration breaks down, the spread becomes a random walk. The Z-score will remain wide, triggering continuous buy/sell orders that result in large drawdowns.

4. **Why run the RTS smoother in backtests but not in production?**

The RTS smoother utilizes future information to refine past estimates. This is optimal for historical parameter calibration but impossible for real-time execution.

Variance break heuristic: A rapid drop in the Kalman variance $P_{t|t}$ suggests the model has converged on a stable relation.

Pipeline Position

Step	Module	Output
5a	<code>spread_builder.py</code>	Initial Spread
5b	<code>half_life.py</code>	$\tau_{\text{half-life}}$ windows
6a	<code>kalman_filter.py</code>	Filter recursion
6b	<code>smoother.py</code>	Backward smoothing
6c	<code>dynamic_beta.py</code>	Full Pipeline orchestrator
7	<code>kalman_forecast.py</code>	Spread projections
8	<code>signal/pair_signal.py</code>	Entry/exit logic